

实验报告-Datalab

常骏 25300180031

本报告为 DataLab 实验的解题记录。DataLab 要求仅使用规定的整数与位运算，在不使用浮点数运算和部分控制结构的限制下，实现整数与单精度浮点数的一系列底层函数。我在本次实验中完成了全部 19 个函数，最终评测得分 110/110。以下按题目顺序整理我的分析过程、关键思路、边界情况处理以及部分失败尝试，重点记录从问题理解到最终通过的完整思考路径。

P1:

比较直接，0x80000000 其实就是最高位从第一位变成了第 32 位，那么就是 $1 \ll 31$ 。

P2:

我们考虑对值分别为 1 和 0 的 a, b 进行 $\sim$ 与 $\&$ 操作，看看能否和 $\wedge$ 结果一致：

例如说 $\sim a \& b$ ，至少在 a, b 相同时是 0，但是 $a = 1, b = 0$ 时是 0。

要和对称的 $a \& \sim b$ 结合， $a = 1, b = 0$ 时是 1。

也就是说如果 $a = b$ ，两个都是 0，但是 $a \neq b$ 时其中一个为 1。

那加起来就可以了！把两个加起来，那么如果 $a \neq b$ ，那么 $\sim a \& b + a \& \sim b = a + b = 1$ ；另一个同理为 a ，相当于 $\sim a \& b + a \& \sim b = a + b = 1$ 。如果 $a = b$ ，那么 $\sim a \& b = \sim b \& a = 0$ ，另一个也是 0，加起来就是 0。这与 $a \wedge b$ 的值一致。

Q: 这里用不了加号。

我们考虑用 $\sim$ 实现 $|$ 类似的功能，完成题目：其实 $|$ 代表着只有两个都是 0 才返回 0，那么其实可以是 $\sim a = 1, \sim b = 1$ ，即当且仅当 $\sim a \& \sim b = 1$ 时才返回 0，其余都是 1。

那么其实 $\sim(\sim a \& \sim b)$ 就会在都是 0 时返回 0，其他情况括号内就是 0，外面就含有 1。

所以 $a | b = \sim(\sim a \& \sim b)$ 从而可以用 $\sim(\sim((\sim a) \& b) \& \sim(a \& (\sim b)))$ 表示结果。

由于 $x = a_n a_{n-1} a_{n-2} \dots a_1, y = b_n b_{n-1} b_{n-2} \dots b_1$ ，所以实际上因为 $\sim$ 和 $\&$ 都只是对于位的操作，可以视为位操作，所以 $\sim(\sim((\sim x) \& y) \& \sim(x \& (\sim y)))$ 实际上就是每一位变成 $\sim(\sim((\sim a_i) \& b_i) \& \sim(a_i \& (\sim b_i)))$ ，每一位上的变化逻辑和 $a_i \wedge b_i$ 一样，那么整体的结果和 $x \wedge y$ 一致。

P3:

首先由于如果是正数要化为 0，那么就考虑用最高位生成的掩码完成这一点：

```
int mask = x >> 31;
```

那么 $x < 0$ 时 $mask$ 有 32 个 1，和任何一个数 $\&$ 都不改变其值；

$x \geq 0$ 时 $mask = 0$ ，和任何一个数 $\&$ 都是 0。

接下来先考虑对负数求其正值：如果 $x < 0$ ，那么 $\sim x + 1 = (-x)$ 。

那么另 $int\ ans = \sim x + 1$ ；然后 $(\sim x + 1) \& mask$ 即可。

这是因为如果 x 大于等于 0， $mask = 0$ ，就返回 0；否则 $mask$ 全 1，返回 $\sim x + 1$ 原值。

P4:

复制一段东西肯定需要掩码，根据题意掩码一般需要八位，那么就考虑使用 0xFF，对它调整。

对于 0 用 0xFF 就行,对于 1 要移动八位,相当于乘以 2^8 。

2: * 2^{16} 16 位

3: * 2^{24} ; 24 位

问题在于 1, 2, 3 对应的移动位数怎么凑。但是 $3 = 2 + 1$, 实际上 3 是 11, 让前面的 1 和后面的 1 都向前移动 3 位就行了。那就变成了 $16 + 8 = 24$ 了!

移动位 $m = \text{src} \ll 3; / \text{dst} \ll 3;$

然后先拷下来 changed_byte,再把它回滚到最前面的 byte,然后再到 dst 对应的位置,最后再把 x 对应的位置替换下来。

经过一定的思考,我一开始使用了如下代码:

```
int copyByteWithin(int x, int src, int dst) {
    int mask_ori = 0xFF;
    int m_src = src << 3;
    int m_dst = dst << 3;
    int mask_dst = mask_ori << m_dst;

    int changed_byte = (mask_ori << m_src) & x;
    int changed_to_0 = (~mask_ori) | (changed_byte >> m_src); //把要改变的byte移到最低位:第0位。
    int changed_to_dst = changed_to_0 << m_dst;
    int changed = (changed_to_dst) & (mask_dst | x);
}
```

但这样 changed_to_0 向前移动时会产生最左边的一部分 0,这样不符合除了改变位其余都是 1 的目的。所以应该另想办法。

其实可以把 mask_ori 移动到 mask_dst 的位置,然后再取反再或的。(~然后|)

改为如下:

```
int copyByteWithin(int x, int src, int dst) {
    int mask_ori = 0xFF;
    int m_src = src << 3;
    int m_dst = dst << 3;
    int mask_dst = mask_ori << m_dst;

    int changed_byte = (mask_ori << m_src) & x;
    int changed_to_0 = changed_byte >> m_src; //把要改变的byte移到最低位:第0位。
    int changed_to_dst = (~mask_dst) | (changed_to_0 << m_dst);
    int changed = (changed_to_dst) & (mask_dst | x);
}
```

核心是先把 x 中原来 src 位置的东西移动到最低位,再移动到 dst 位,然后把周围其他位全部变成 1:

111111...src11111111 【在 dst 位更改】

然后把需要更改的 x 的 dst 位改成 11111,其他位保持原值。

利用 $1 \& a = a$,两个变量相互&,就得到需要的答案值。

P5:

我们没有别的右移的方法,所以显然唯一的方法就是先算术右移,然后如果算术右移后多了若干位 1,(例如移动 n 位,那么可能从最高位开始的前 n 位都变成了 1:111111...1xxxxxxx, n 个多的 1) 那么目标就是把这 n 个 1 除掉为 0。

自然考虑掩码,如果有 000...0(n 个 0)11111111...1111 和上面的数相互&就可以得到想要的逻辑右移值。

问题在于，我们需要计算 $32 - n$ ，它等于 $31 - n + 1$ ，而 $31 - n$ 就是把 n 中为 1 位换成 0，其他的换成 1 就可以。

这个解法看似是对的，但是如果 $n = 0$ ，会触发未定义行为（ $1 \ll 32$ ，超过了 31）；而且 $\text{int mask} = (1 \ll ((31 \wedge n) + 1)) - 1$ ；使用了 -1，违反了系统的要求。

第二次尝试：

我们需要创造 $000\dots 0$ (n 个 0) $1111111\dots 1111$ ，那么就考虑先创造其反面： 1111111 (n 个) $00000\dots 000$ ，这可以通过最高位 1 ($10000\dots 00000$) 移动 $n - 1$ 位得到。

问题就在于表达 $(n-1)$ 。

问题在于 $n=0$ 时相当于 $10000\dots 000000 \gg -1$ ，这是题目要求不允许的，实际操作也会取 mod 完成不了，所以要考虑替代。

通过一定时间思考，发现了灵感：可以把 -1 分解成先 $\gg n$ ，再 $\ll 1$ ，好处在于：

如果 n 是 0，这样一步一定会保持 $10000\dots 000000$ 不变然后再把 1 移走，变成 0。这样~后变成 $1111\dots 11111$ ，掩码后保持 x 不变。

$N \geq 1$ 时则也达到了目的，得到 $111\dots 1$ (n 个) 000000 。这样就可以取反掩码，保留前 n 位的后所有位，把前 n 位的补码消除为 0。

P6:

实际上就是先用掩码 $0x0F0F0F0F$ 和 x 取 &，得到 x 每个 byte 的低四位（其他的都变成 0），然后向高处移动 4 位；另外一边也是，把另一边截取的高四位向右移动 4 位，变为低 4 位。把这两种方法得到的结果用 | 并起来，就是高低位交换后的值。

这里注意另一边截取高 4 位后要右移 4 位，这个必须是逻辑右移动，就需要上一题代码。但这一题的优势在于这里的 n 是确定的，不需要对边界检查，可以把上一题的逻辑简化为：

P7:

这个问题很难，比较刁钻，我思考了十几分钟想到了这样做：

首先第一位前面的形式相当于 $\dots xxx0111111$

那么对这个值加 1 得到 $\dots xxx1000000$ ，后面的 x 和未加的时候值一样，那么对他们取 ^ 就会得到： 000001111111 ，其中最高位 1 的位就是第一个 0 的位，对其 $(x+1) \gg 1$ 就可以。
[需要四步]

（这里如果第一低位 0 根本不存在，就会返回 0，取 ^ 就是 $1111\dots 11111$ ， $+1 \gg 1$ 就是 0，可以反映出最低 0 位不存在。）

得到了之后填充最低 0 位再加 1，第二低的 0 位变成 1，重复上述操作，就可以得到答案。

（对于只有一个 0 位的数，第一次操作得到最低 0 位为 1，其余为 0 的数。这个数加上原来的 x 后就会把最低 0 位填上，那么第二次重复第一次的步骤得到的 $(x+1) \wedge x = 0$ ，这个东西 $+1 \gg 1$ 还是 0。

稍等！ $+1 \gg 1$ 对于第一次补最低位，变成 $0x7FFFFFFF$ 的数容易产生边界情况，这时

+1 之后>>1 可能导致溢出，然后得到 0，不符合要求。所以我们改成 $(x \gg 1 + 1)$ ，这样最高位肯定会控制在当前数最低 0 位这一位，不会产生溢出。

但是这种策略有致命缺陷！！0x7ffffff 与 0xffffffff 经过这种策略计算一轮后，算出来的值是一样的，且无论前面这种还是后面这种策略算出来都是 0。

这样没办法把 0x7ffffff 和 0xffffffff 区分开来。）

还有一个问题在于两次使用这个方法就需要 8 步了。

我又思考了一段时间， $x = \dots\text{xxx}0111111$ 与 $\dots\text{xxx}1000000$ 可以这样压缩得到最低 0 位的位信息：通过 $(x + 1) \& \sim x$ 即可。

这种方法不仅消除右边的相同项，而且由于 $x+1$ 的项恰好是 x 的取反，所以恰好相等，&后保留。这个方法很好，而且：

对于 0x7ffffff:得到的是 0x80000000;

对于 0xffffffff 得到的是 0x00000000，产生了区别。

这次能产生区别的本质原因：因为我前面的方法得到的是最高位对应的 1111。。。111 序列（对于 0x7ffffff，0xffffffff，两个不一样，但是扔掉溢出项就一样了。）后面的方法得到的是最高位为 1，其余为 0 的掩码，没有了中间项，一个掩码明显溢出了，返回 0；另一个对应 10000000，产生了明显的不同。

我们写代码如下：

```
int secondLowestZeroBit(int x) {
    int firstmask = (x + 1) & (~x);
    int x_complmented = x + firstmask; //补上最低位的0
    int secondmask = (x_complmented + 1) & (~x_complmented);
    return secondmask;
}
```

边界情况：

1 如果传进去的就是 0xFFFFFFFF，其最高 0 位为 1 的 mask 就是 0，这和我们认为的一致。

2 如果传入的只有一位不是 0，那其实它没有第二个 0 位，我们认为不需要 mask，返回 0。那么其实它会使得 `x_complmented` 变成 -1，这样问题转化为求其填上最低位 0 后的值的最低 0 掩码，我们认为它就是 0，而上述代码会使得掩码为 $(0 \& 0) = 0$ 。

所以边界情况返回值正确，这个方法成立。

P8:

CSAPP 书上有一道（我之前已经做到的）汇编语言转 C 语言的题目，里面就有其实现算法。大致是这样：用 1 与 x 的二进制表示的每一位做 $\wedge$ ，如果遇到 0， $0 \wedge a = a$ ；如果遇到 1：

$1 \wedge a = (\sim a)$ ， $1 \wedge 1 = 0$ ， $1 \wedge 0 = 1$ 。从而如果 x 的二进制偶数个 1，其他的 0 在与变量 $\wedge$ 时不改变变量，而 1 会偶数次取反变量，相当于没有取反，回到 1。

而如果 x 的二进制奇数个 1， $(2k+1)$ 那么每两个 1 会让 1 变成 0，然后再变成 1，重复 k 次后，最后一个 1 会 $1 \wedge 1 = 0$ ，此后（以及此前的 0）都只会保持原来值不变，所

以最终是 0.

这就达成了题目的要求。需要说明的是这需要 for 循环的逻辑，但是只有 32 位，也可以直接展开 32 位循环。

然而：

```
signal = signal ^ x;  
x >> 1;
```

每一步都需要至少 2 个操作符，那么其实 $2 \times 32 = 64$ 超过了 56！所以其实是有问题的。

我们考虑压缩其中的一些步骤，比如 1 位改成 2 位去思考。

如果原来的两位是 11，k 位 1 时：两个 1 分别与 m 与 n 位 1 进行了 ^ 操作。

k 为偶数时，可能：m 偶 n 偶，那么 11 保持 11；m 奇 n 奇，11 为 00。

k 为奇数：mn 一奇一偶：01 或者 10。

所以其实最后两位如果都是一个值，那么 k 为偶数；如果是不同值，则 k 为奇数。

因此，理想的标识符应该是 $\sim((x \& 2)^{(x \& 1)})$ 。

这种情况只需要 $2 \times 16 + 4 = 36$ 个 operator，明显满足。

另外输出格式需要改进：

```
int ans = (~(((signal & 2) >> 1) ^ (signal & 1))) & 1;  
return ans;
```

说明：1 原来的 $\sim((\text{signal} \& 2) \wedge (\text{signal} \& 1))$;

本意是把第二位的值和第一位做异或，但是在这里实际上第二位的值留在第二位，第一位的值留在第一位，做不了异或。改成： $\sim(((\text{signal} \& 2) \gg 1) \wedge (\text{signal} \& 1))$

2 而且 $\sim$ 后虽然所得最后一个值 a_0 变成了 $\sim a_0$ ，但是括号内的 $00000000 \dots a_0$ 变成了 $111111111 \sim a_0$ ，我只想要 $\sim a_0$ 。所以外面还要加一层 $(xxx) \& 1$;

然后就得到正确函数。

P9:

其实很简单，把后 $32-n$ 位用前面的函数里一样的方法逻辑右移，移动到前面。前 n 位移动到后面，然后 | 得到答案。

P10:

比如说一个数的 2 进制表示是 $a_m a_{m-1} a_{m-2} \dots a_n a_{n-1} a_{n-2} \dots a_1$ ，那么其实问的就是 $a_n a_{n-1} a_{n-2} \dots a_1$ 离哪个更近，怎么判断 + 舍入。

我的考虑是这样：把 $a_n a_{n-1} \dots a_1$ 这一段截出来单独计算，如果 $+ 01111111 \dots 111$ ($n-1$ 个 1) 导致第 $n+1$ 位 (2^n 代表位) 出现了 1，那么说明： $a_n a_{n-1} \dots a_1 \geq 1000000000 \dots 0001$ 。

对于 $\leq 011111111111...1111$ 的情况, 无论如何都小于 2^n , $n+1$ 位: 0。

这里记对 x 截断后一系列操作后的数为 b , b_{n+1} : 第 $n+1$ 位。

这里如果 $b_{n+1} = 1$, 说明截断的值足够大了, 需要向上进位; 否则截断不够大, 保持 a 的更高位原样。

对于中间值可以利用原来 x 的第 $n+1$ 位 b_{n+1} , 由于最终要靠近偶数: 那么如果 b_{n+1} 本身偶数, 那么就希望舍; 否则为 1 希望入 (这样就加 1 变成了 0) 那么可以

$a_n a_{n-1} \dots a_1 + 01111111 \dots 111$ 再+ b_{n+1} , 那么特殊情况下:

1 b_{n+1} 偶, 那么加起来为 $1111111111 \dots 1111$, $a_{n+1} = 0$, 不入, 得到的是最近的偶数。

2 b_{n+1} 奇, 那么加起来为 10000000000 , $a_{n+1} = 1$, 入, 得到的是最近的偶数。

如果 $a_n a_{n-1} \dots a_1 \geq 1000000000 \dots 0001$. 那么理论上就要入, 实际上即便 b_{n+1} 最小, 加上另一段肯定也会出现 $n+1$ 位为 1, 那么就会提示计算机 $a_{n+1} = 1$, 应该增加。

另一种情况同理, 无论如何 $n+1$ 位是 0, 那么就不会增加原来 $n+1$ 位的值。

使用此思路编写后, 此题满分通过。

P11:

如果 x, y 同奇偶, $(x+y) \gg 1$ 就可以了, 关键在于不是同奇偶的情况。

这种情况下, 如果 $x > y$, 那么向上舍入, $x+y$ 向下舍入 $/2$ 的值 + 1;

否则 $x \leq y$ 向下舍入, $(x+y) \gg 1$ 自动就是向下舍入 (最低一位 1 除 2 的值直接被抛弃了)。

具体由于要考虑溢出, $x > y$ 的判断很复杂:

$x > y$ 两种情况: $y - x$ 最末位为 1 或 $y - x$ 为正时 y 为负且 x 为正, 此时才是 $x > y$ 。

但是后面一种情况基于 $y < 0, x > 0$, 这时只论 $x > y$ 肯定是对的。

那么其实条件改成 $(y - x < 0) \mid (y < 0) \ \&\& \ (x > 0)$; 就行了。

这是因为 $y < x$ 时只有两种情况, 使得 $y - x$ 算出来反而大于 0 的情况只会在 $y < 0, x > 0$ 的情况发生, 那么就能推出上述条件式都是 1。

看上述条件式是否包含别的情况: 右边的满足肯定是 $y < x$; 左边满足还有例外情况: 如果 $y > x$ 但是 x 过分小了, 那么就会出现溢出然后变成负的 $y - x < 0$ 。

所以改成 $((y - x < 0) \mid (y < 0) \ \&\& \ (x > 0)) \ \&\& \ \sim((y > 0) \ \&\& \ (x < 0))$

注意: 不能用 $y - x$; 那么差值表示为 $y + \sim x + 1$ 即可。

把上述思路写成代码后, 满分通过。

P12:

其实是 $(x-a)(x-b) \leq 0$ 的检验。

我们考虑 $(x > a) \wedge (x > b)$ 如果等于 1 那么只可能:

1 $a < x \leq b$,

2 $b < x \leq a$

这两种可能都对应 x 在 interval 内, 但是漏掉了 a 小时 $x=a$, b 小时 $x=b$ 。

也就是应该 $\mid$ 上 $x==a \mid x==b$

此时当 x 在 interval 内时, 返回 1; 且 x 在 interval 外时, 要么 $x > a, b \mid 0 \mid 0 = 0$

要么 $x < a, b \mid 0 \mid 0 = 0$

所以是当且仅当，这个方法是成立的。

（判断 $x > a$ 的方法在 P11 里已经得出了，这里复制 P11 的方法即可，算符有 48 个，非常充足，不需要考虑限制问题。）

另外 $x == a$ 不能直接用，需要代替：

我们使用 $!(x \wedge a)$ 表示 $x == a$ ，如果 $x = a$ 异或就是 0，返回 1

如果 $x \neq a$ ，那么至少有一项不同， $x \wedge a \neq 0$ ， $!(x \wedge a) = 0$ ；

这和 $x == a$ 相同。

P13:

我们主要考虑溢出的情况：

实际上要算的无非就是 $x \ll 2 + x(x * 4 + x = x * 5)$

那么关键是何时溢出？可以知道，如果要正溢出：

那么要么 x 自己的倒数 2 位【第 31，30 位】是 1，要么就是 $x * 4 \leq 2^{31} - 1 < x * 5$ ，也就是刚刚好 $x * 5$ 使得 $x * 5$ 的第 32 位是 1（可以估计出最大值不超过 100111...111(2 进制，32 位) 第 32 位不会进一步溢出。）

负溢出：

会复杂一些，先考虑判定 $x * 2$ ， $x * 4$ 溢出：如果负数是 101xxxx/110xxxxxx，那么负数最大是 $-1 - 2^{29}$ ，其四倍肯定大于 -2^{31} ，肯定溢出了！要么 $x * 4 \geq -2^{31} > x * 5$ ，其实就有负溢出， $x * 5$ 实际上变成了大于 0 的数，第 32 位为 1。

用逻辑运算符把溢出的情况整理好即可。

```
int posi_flow = (~x_sig) & (((x >> 30) & 1) | ((x >> 29) & 1) | sum_sig); //表示正溢出，1为正溢出。
int nega_flow = x_sig & (((x >> 30) ^ 1) | ((x >> 29) ^ 1) | (~sum_sig));
```

这里耗费的算符太多，我简化一下，第 31 位，30 位有一个 1 可以简化为 $\sim(!((x >> 29) \& 3))$

另一部分要求只要有 0 就行，那么相当于 $\sim((x >> 29) \wedge 3)$

修改的其他处：

```
int sum = (x << 2) + x; //但还不是最终答案。
int x_sig = (x >> 31) & 1;
int sum_sig = sum >> 31;
int posi_flow = (~x_sig) & ((~(!((x >> 29) & 3))) | sum_sig); //表示正溢出，1为正溢出。
int nega_flow = x_sig & ((~(!((x >> 29) ^ 3)) | (~sum_sig));
```

```

int mul5Sat(int x) {
    int sum = (x << 2) + x; //但还不是最终答案。
    int x_sig = x >> 31;
    int sum_sig = sum >> 31;
    int posi_1 = !((x>>29) & 3);
    posi_1 = posi_1 + ~0;
    int nega_0 = !((~(x>>29)) & 3);
    nega_0 = nega_0 + ~0;
    int posi_flow = (~x_sig) & (posi_1 | sum_sig); //表示正溢出, 32位均为1为正溢出。
    int nega_flow = x_sig & (nega_0 | (~sum_sig)); //类似上一句。
    int intmin = (1 << 31);
    int intmax = ~intmin;
    int temp = sum ^ ((intmax ^ sum) & posi_flow);
    int ans = temp ^ ((intmin ^ temp) & nega_flow);
    return ans;
}

```

具体的实现逻辑比较复杂，只列出我探索这个写法过程中思考的主要几点：

关于无分支条件选择，在不允许使用三目运算符的前提下，通常利用掩码来实现。常规做法是 `int mask = ~(cond - 1); int result = (mask & a) | (~mask & b);`，但此写法需要用到减号，且运算符多了一个。我于是思考了更节俭的解法。

ICS 书上蓝色部分有一个 $b \wedge (a \wedge b) = a$ 的等式，结合这个等式，可以发现 $b \wedge ((a \wedge b) \& \text{mask})$ 在 `mask` 全为 1 时就是 $b \wedge (a \wedge b) = a$; `mask = 0` 时就是 `b`，可以产生条件语句的效果，且很节省。所以采用了它。

在具体到 `mul5Sat` 的溢出判断时，我们希望的是如果正溢出，那么 `posi_flow` 全部 32 位都是 1；否则全部都是 0。负溢出同理。那么这里我就需要保证得到 `posi_flow` 的中间变量也只有“32 位全部为 1”和“均为 0”2 种状态，不存在其他状态。

```

int posi_flow = (~x_sig) & (posi_1 | sum_sig); //表示正溢出,
int nega_flow = x_sig & (nega_0 | (~sum_sig)); //类似上一句。

```

`(~)x_sig` 和 `sum_sig` 可以很容易地保证只能是全 1 或者全 0，所以问题的关键在于保证中间检测第 31 位和 30 位的 `posi_1, nega_0` 必须全 1 或者全 0。

原理上，只有 31, 30 位的信息通过掩码只能得到 1-2 个位是 1 或者 0，那么我们要把 1/0 扩展到所有位只有两种方式：

1 这一位是最高位，`>>31`，那么 1 转为全 1，0 转为全 0。然而 31, 30 位不是最高位，不可行。

2 这一位在中间，那么 `!(中间数 & ...1...)` 就是 0，0 取反就是全 1，即 `~!(中间变量 val & 0001000)` 就可以。此时如果 `val` 的对应位位 0 就得到 `!0 = 1`，取反会得到 `1111111111....10` (和我们希望的 `000000...0000` 差别很大)。那么就需要在这种情况下增加 2，使得 `0-->0000...000`

另一种情况(中间变量 `val & 0001000 != 0` 的情况)则产生 `~0 = 1111111111....1111`，那么已经得到想要的结果，不需要再加。那么其实每次都增加 `2 * (!(中间变量 val & 0001000))` 即可产生想要的结果：`0-->0000...000`，`1->1111111111....1111`。

部分代码如下：

```
int posi_1 = !((x>>29) & 3);
posi_1 = (posi_1 << 1) + (~posi_1);
int nega_0 = !((~(x>>29)) & 3);
nega_0 = ~nega_0 + (nega_0 << 1);
int posi_flow = (~x_sig) & (posi_1 | sum_sig); //表示正溢出, 3
int nega_flow = x_sig & (nega_0 | (~sum_sig)); //类似上一句。
```

这是尝试过程中写的代码，最终版在此基础上有修改。

对正数，如果第 31, 30 位有 1，这一个变量就必须是 1111111111.....1111111,表示会溢出；对负数，如果第 31, 30 位有 0，这一个变量就必须是 1111111111.....1111111,表示会溢出。

所以满足，但是使用符号太多，我绞尽脑汁，仍然难以想出简化方法（实际上总符号减少一个就能满足要求了）我于是适度询问了 AI：

我的原始办法是 `int posi_1 = !((x>>29) & 3); posi_1 = (posi_1 << 1) + (~posi_1);` 实际上 `posi_1 = (posi_1 << 1) + (~posi_1);` 的实际作用就是：以 `posi_1` 的上一阶段值为输入，输入 0 变成全 1，输入 1 变成全 0。用 ~ 实现，仅对 0 可以，对于 1 不能直接变成这样就进行凑，所以才使用了这个复杂的方法，完成这个操作。

AI 给我的核心顿悟是不要从位上看，从数学上看，上述的输入-->输出操作其实是 0 转为 -1, 1 转为 0 的一个函数。那么这个函数的表达式也可以表达为 `posi_1 - 1`，即 `posi_1 + ~0`，这样每个算式都可以减少一位（运算符）。这个操作成功将 ops 从 31 压缩到了 29（修改两处）也让我明白了一个函数实现的问题，从位运算的角度和数学的角度有不同的解答，要多角度分析，得到最优的方法。

P14:

主要是这里要加三个数，我们也许可以考虑对三个数比较大小，然后分别排序？毕竟赋值操作是被允许的。但是思考了一段时间后，发现排序后也无法确定哪些小于 0，哪些大于 0，那么其实排序没什么用。

我们于是考虑别的问题：同号时怎么判断溢出？实际上只要 `x + y` 溢出或者 `(x + y) + z` 溢出（即使前者不溢出）就能判断溢出了。

实际上可能不需要进行很强的分类讨论：先判断 `(x + y)` 是否溢出就行了，如果没有溢出判断 `+z` 是否溢出：

`x + y` 本来 ≥ 0 , `z` ≥ 0 , 那么加起来首位为 1 就是溢出。

`x + y` 本来 < 0 , `z` < 0 , 加起来首位为 0 就是溢出。

但是如果 `x+y` 就溢出了呢？那么分类：1 正溢出：

那么如果 `+z` 又产生负溢出，那么就可以返回正常值：这是因为这说明 `x + y - 2^32 + z < -2^31 ==> x + y + z < 2^31`。另一方面也说明算出来的一个在 int 范围内的值是 `x + y + z` 先 -2^{32} 再 2^{32} 后得到的值，说明 `x+y+z` 在范围内。

如果 `+z` 之后保持不溢出（这里也不可能正溢出），这说明 `x + y - 2^32 + z >= -2^31`, 也就说明了 `x+y+z >= 2^31`。也就是和溢出！

2 负溢出 差不多:

如果 $+z$ 正溢出, 那么说明 $x+y+z + 2^{32} - 2^{32}$ 得到 $x+y+z$, 且可以被表示为 `int` 范围内的值, 这说明 $x+y+z$ 就在 `int` 的整个范围以内, 总体不溢出。

如果 $+z$ 不溢出, 说明 $x + y + 2^{32} + z < 2^{31}$, 也就是说 $x + y + z < -2^{31}$, 这种情况下, 说明最终得到的值小于 `INT_MIN`.

根据上述的分析写代码即可, 最终 `op` 用了 40/52, 通过。

```
int classifyAdd3(int x, int y, int z) {
    int x_sig = (x >> 31) & 1;
    int y_sig = (y >> 31) & 1;
    int z_sig = (z >> 31) & 1;
    int half_sum = x + y;
    int half_sum_sig = (half_sum >> 31) & 1;
    int p_flow = ((~x_sig) & (~y_sig)) & half_sum_sig;
    int n_flow = (x_sig & y_sig) & (~half_sum_sig);

    int sum = half_sum + z;
    int sum_sig = (sum >> 31) & 1;
    int p_flow_sum = (~half_sum_sig) & (~z_sig) & sum_sig;
    int n_flow_sum = (half_sum_sig) & (z_sig) & (~sum_sig);

    int half_flow = p_flow | n_flow;
    int pos_part = ((~half_flow) & p_flow_sum) | ((p_flow) & (~n_flow_sum));
    int neg_part = ((~half_flow) & n_flow_sum) | ((n_flow) & (~p_flow_sum));
    int ans = pos_part + ~neg_part + 1;
    return ans;
}
```

P15:

计算 $f * 3/2$, f 其实是单精度 `float` 的 32 位表示。

我考虑的方法是保留 `exp` 的位数, 并且考虑把 `frac` 左移一位, 自增, 这样在 `exp` 不变的情况下其实得到的约为 $2^{(exp - 127)} * (1 + frac + 0.5frac + 0.5)$ 。

那么要考虑几种情况:

- 1 加起来没有溢出也没有多的小数位, 那么就保留其值。
- 2 加起来溢出了, 但是没有多的小数位, 那么就考虑 `exp` 位增加 1, 然后整体左移动, 再舍入。
- 3 加起来没有溢出, 有多的小数位, 那么就看看奇偶考虑舍入。
- 4 加起来溢出了, 有多的小数位, 那么就先左移, `exp` 加 1, 然后再看多余位的情况, 这时多余位肯定不会是 10, 就可以直接比较大舍入。

另外还有一种需要处理的特殊情况: 传入 `NaN` 返回原值这一分支, 应该一开始就判。(可以和正负无穷一起判, 因为 $1.5 * \text{正无穷/负无穷}$ 返回的还是原值)。

```

unsigned new_frac = frac << 1;
unsigned sum = (frac << 1) + frac + (1 << 23);
if ((sum >> 25) & 1) {
}
else {
    unsigned sig = sum & (sum >> 1) & 1;
    unsigned new_frac = (sum >> 1) + sig;
    if ((new_frac
}

```

我们考虑极端情况，也就是 new_frac 刚好溢出的情况，这需要原来的 sum（把 frac + 0.5 + frac*0.5 用 24 位表示的值）24 位都是 1，也就是 frac + (0.5 + frac*0.5) 没有进位（有进位最早进位的位出现 0，不符合），从而 frac 和 0.5 + frac*0.5 里面的 1 的个数之和就是新的 24 位值的 1 的个数。设 frac 中 a 个 1，那么 $24 = 2 * a + 1$ ，这里显然没有 a 满足，所以加起来无论如何都不可能出现 24 个 1，所以 24 位 1 的和值实现不了！这也就是说要么高 23 位含 1，要么最后一位是 0。后者不需要舍入，前者由于高位存在 0，即便舍入加 1 也不会溢出。

所以不需要判溢出！只需要判一个：小数最末位是否进位。

中间还遇到了错误：ERROR: Test floatScaleThreeHalves(0[0x0]) failed...
 ...Gives 4194304[0x400000]. Should be 0[0x0]

这里说明没有对非规格化值单独处理，我们对之单独处理。注意非规格化值的 frac*1.5 溢出的情况，可以视作溢出的 1 变成了 exp 位的第一个 1，因为若 $\text{frac} * 1.5 = (1.x) * 2^{(1-\text{Bias})}$ ，而视作溢出的 1 变成了 exp 位的第一个 1 时，这个 32 位码也被解释为（忽略符号位） $(1.x) * 2^{(1-\text{Bias})}$ ，所以可以把溢出视为 frac 为自然溢出到 exp 位，也即可以把溢出数的二进制视为目标数表示的前 31 位。

那么 ans = 溢出数 | 符号位 即可。

汇总成代码后，编写调试，最终通过。

P16:浮点数的近似：

关键是算出来小数部位是哪一部分。

如果 exp 位的值 E: $(E - 127) \geq 23$ ，算出来一定是整数，这个时候所表示的数肯定是整数，所以直接返回。

如果 $E - 127 \leq -2$ ，那么肯定代表的绝对值 $< 1/2$ ，肯定舍入为 0，那么其实返回符号位+31 个 0 就行。（所以其实非规格化值也不需要考虑了，都作为 0 返回即可。）

那么我们只考虑 $-1 \leq E - 127 \leq 22$ 。

```

unsigned e = exp - 127;
unsigned ans;
if (e >= 23 || ((exp >> 23) == 0xFF))
    return uf;
}

if (e <= -2) {
    return sign;
}

if (e == -1) {
    if (!frac) {
        return sign;
    }
}

```

这里遇到了一个细节问题：e 不能是 unsigned，这样 e 小于 127 时减去 127 会溢出，增加 2^{32} 次方变成非常大的数，e 比较小时都不会触发比较条件。所以需要修改。

```

frac = frac << e; //这样frac第24位以后的位就是浮点数二进制表示中小数点左边的部分。
if (((frac & m3) == (unit >> 1) && (frac & unit)) || (frac & m3) > (unit >> 1)) {
    frac += unit;
}
frac = frac & (temp);
frac = frac >> e;
//注意：例如frac = 11111...11111 全1的情况，化整数后需要对其四舍五入。
//这体现在frac舍入后，回到1-23位时，如果最高位又出现了1，那么就说明表示(1 + frac)表示数>=2。
//实际上只可能等于2。

if (frac & (1 << 23)) {
    frac = 0;
    exp += 1;
    ans = sign | exp | frac;
    return ans;
}

```

修改好上一个 bug 后发现这里还有另一个 bug：由于我们是根据将 frac 变成其原来代表的小数表示，得到整数部分最后一位。上面的代码一般无问题，但是如果 2^e 中 $e=0$ ，那么实际上表示的数字绝对值是 $1.\text{frac}$ ，但是上述代码当成 $0.\text{frac}$ 处理， $\text{frac}=5000\dots0000$ 时当作整数位是 0 化为 0，但是实际上是 1.5 应当化为 2，然后再变成 $e=1$ ， $\text{frac}=0$ 。

问题来源于对 float 代表的数字的表示不完整，从而对于其最小整数位可能读取有误。所以应当在一开始就在 frac 的第 24 位赋予 1，使得 frac 能够代表完整的小数部分值。

问题 2：e = 21 时移动 21 位，但是总共 32 位，位不够，需要转变方式。

P17:把 int 值表示成 float，输出这个 float 的位级表示。

首先由于 x 的 float 表示的前 31 位只由 x 的绝对值决定，通过 ^ 与 & 组成的位级条件语句得到 x 的绝对值：

```

int sign = (1 << 31) & x;
int m1 = sign >> 31;
unsigned minus_x = ~x + 1;
unsigned abs_x = x ^ ((minus_x ^ x) & m1); //通过^(^)&的条件语句得到x的绝对值。|

```

$\text{Abs}(x) \leq 2^{31}$ ，那么 x 最多需要 exp 为 $127 + 31 = 158 < 255$ ，实际上不可能溢出，所以不

考虑溢出。最少需要 $127 + 0 = 127$ 。

但是注意 0 需要的 `exp` 是全 0，远小于 127，所以这个要特殊处理。

一开始未考虑舍入，所以出现报错：

ERROR: Test float_i2f(-2147483647[0x80000001]) failed...

...Gives -822083585[0xceffffff]. Should be -822083584[0xcf000000]

前者小数部分舍入后，就是小数部分为 1，整体为 2，这个 2 需要进入 `expr` 部分，所以 `expr` 相比于未舍入的值 10011101 增加了 1，为 10011110，整体显示为 11001111 而后都是 0，即 0xcf000000。

这说明我得到的值舍入就是正确值，于是增加舍入代码。

当然舍入之后，可能出现 24 位的 1111111111...11111 （这是 `x` 的浮点表示中的小数部分 `frac` 与它前面的 1，即 $1 + 23$ 位的 `frac`）--> 1000000000000，其中 1 变为第 25 位的值。从而导致小数部分溢出为 2，这里需要一段代码判断这种及其特殊的情况，并且将 2 降为 1（`frac = 0`），`exp` 自增 1，实现代码如下：

```
if ((1 << 24) & abs_x) {
    expr++;
    abs_x = 0;
}
```

这里 `abs_x` 实际上存储的是 `x` 的 float 表示中 `frac` 部分的值，这里肯定是 0。

汇总起来后写出代码，可以跑通满分。

P18:

考虑暴力，但是暴力明显使用的代码太多， $32 * 2 = 64 > 60$ ；（一般需要的比 2 多！）

考虑 2 位一次的暴力，那么其实可以用 $((x \gg 2) \& 3 - ((x \gg 2) \& 2) \gg 1)$ 算出 `x` 某两位的 1 个数。问题在于仍然需要 $5 + 2$ （减号换成~+）个 operator， $7 * 16$ 严重超标！

所以要思考别的方法。

我思考到这样一个做法：可以计算： $(x \gg 1) + x$ 。

第 $2k+2$, $2k+1$ 位如果是 `ab`, 那么其实移动后就是 `xa`, 这里把 `x` 掩码变成 0，加起来后就是 $0b + 0a = (a + b)$, 就是第 $2k+2$ 与 $2k+1$ 的 1 的和。

我们考虑用同样的方法移动 2，掩码，加起来，得到 8 个长度为 4 的块，每个块里的值就是一开始 `x` 在这个块的含 1 量。

以此类推，得到四个长度为 8 的块，再合并为 2 个 16 个的块，最后完全合并，结束。

（注意：前两次要用逻辑右移，第三次开始由于最右边的块最大值 `n`，充满不了最后一位，所以逻辑右移就行。）

主要是形如 00110011... 的掩码怎么构造。我们先构造 01010101，就是 0x33；然后把这个向左移动，加起来得到完整的掩码。其他几个也类似。

具体细节不表，代码如下（已跑通）：

```

int bitCount(int x) {
    int mask_4bit = 0x33 | (0x33 << 8) | (0x33 << 16) | (0x33 << 24);
    int mask_2bit = (((mask_4bit >> 1) & mask_4bit) << 1) + mask_4bit;
    int temp = (x & mask_2bit) + ((x >> 1) & mask_2bit);
    temp = (temp & mask_4bit) + ((temp >> 2) & mask_4bit);
    int mask_8bit = 0x0F | (0x0F << 8) | (0x0F << 16) | (0x0F << 24);
    temp = (temp & mask_8bit) + ((temp >> 4) & mask_8bit);
    int mask_16bit = 0xFF | (0xFF << 16);
    temp = (temp & mask_16bit) + ((temp >> 8) & mask_16bit);
    int mask_32bit = 0xFF | (0xFF << 8);
    int ans = (temp & mask_32bit) + (temp >> 16);
    return ans;
}

```

Mask_2bit (1 位合并为 2 位的掩码) 这么做是因为它相比于 $0x55 | (0x55 << 8) | \dots$ 少了一个 operator, 这样一共正好 40 个 operator。

P19:

一样的 2->4->8->16->32 的思路: 两位先调换, 然后 4 位, 8 位.....
这次的限制在于只有 34 个运算符。

```

int bitReverse(int x) {
    // 掩码构造 (14 个算符)
    int m1 = 0x55 | (0x55 << 8);
    m1 = m1 | (m1 << 16);           // 0x55555555

    int m2 = 0x33 | (0x33 << 8);
    m2 = m2 | (m2 << 16);           // 0x33333333

    int m3 = 0x0F | (0x0F << 8);
    m3 = m3 | (m3 << 16);           // 0x0F0F0F0F

    int m4 = 0xFF | (0xFF << 16); // 0x00FF00FF (2 个算符)

    int m5 = 0xFF | (0xFF << 8);    // 0x0000FFFF

    // 5 轮分治交换 (23 个算符)
    x = ((x & m1) << 1) | ((x >> 1) & m1);
    x = ((x & m2) << 2) | ((x >> 2) & m2);
    x = ((x & m3) << 4) | ((x >> 4) & m3);
    x = ((x & m4) << 8) | ((x >> 8) & m4);
    x = (x << 16) | ((x >> 16) & m5);

    return x;
}

```

注: 变量和程序里的不一样, 这是简单化表示, 程序里为了清晰变量名不一样。
这样已经需要 $16 + 24 = 40$ 个 operator 了, 要把 6 个压榨出了。

主要是 5->3 有没有办法，这里直接加和移动都很难，
我们考虑一个生成多个：先出现 $0x11111111 = \text{base}$ 然后

```
int base2 = base << 1;
```

```
int mask1 = base2 + base;
```

```
int mask2 = base2 + mask1;
```

这样生成 $0x55555555$ 和 $0x33333333$ 的算法压缩成了 7 个算符，但是还需要 39.

思考一段时间后想到： $\text{int } m5 = 0x0000FFFF$, 那么 $m4 = (m5 << 8) \wedge m5$;

而且很巧的是 $m3 = (m4 << 4) \wedge m4$使用这种思路即可。

解答（已通过）：

```
int bitReverse(int x){
    int m5 = 0xFF | (0xFF << 8);
    int m4 = m5 ^ (m5 << 8);
    int m3 = m4 ^ (m4 << 4);
    int m2 = m3 ^ (m3 << 2);
    int m1 = m2 ^ (m2 << 1);

    //以下为从小到大分块转换顺序过程。
    int temp = ((x & m1) << 1) + ((x >> 1) & m1);
    temp = ((temp & m2) << 2) + ((temp >> 2) & m2);
    temp = ((temp & m3) << 4) + ((temp >> 4) & m3);
    temp = ((temp & m4) << 8) + ((temp >> 8) & m4);
    temp = (temp << 16) + ((temp >> 16) & m5);
    int ans = temp;
    return ans;
}
```

总结：

本次实验历时约三整天，在反复探索与调试中完成全部 19 个函数，最终评测得分 110/110。过程中经历了多次失败与修正，例如整数运算中操作符限制的反复优化、浮点数非规格化值与舍入边界的细致处理、分治掩码压缩的多种尝试等。通过本次实验，我深入理解了补码、IEEE 754 浮点数编码以及位运算在系统底层的应用，掌握了从问题分析、思路推导到代码实现与调试验证的完整流程。同时也体会到，在严格约束下寻求简洁高效的解，需要数学直觉与工程耐心的结合。